Object-Oriented Programming

Lecture 5 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 What is object-oriented programming? 2

2 Classes and objects 3
2.1 Terminology . . . 3
2.2 Defining a class . . . 4
2.3 Attribute references and instantiation . . . 4
2.4 The constructor `__init__()` . . . 5
2.5 Example: a `Car` class . . . 6
2.6 Understanding `self` . . . 8
2.7 Printing objects: `__str__()` and `__repr__()` . . . 10

3 Class variables and class methods 11

4 Data abstraction and encapsulation 14
4.1 Data abstraction . . . 14
4.2 Encapsulation . . . 15

5 Composition 17

6 Inheritance 18
6.1 Overriding methods and `super()` . . . 21

7 Polymorphism 26

8 Multiple inheritance 27

Summary 31

Exercises 32
Exercise 1: A `Rectangle` class . . . 32
Exercise 2: Shapes and polymorphism . . . 32
Exercise 3: Driving the car . . . 32
Exercise 4: Composition — a cantilever beam . . . 32

Further reading 33

Learning objectives

By the end of this lecture, you should be able to:

- explain the ideas of classes, objects, attributes and methods;
- define your own classes with a constructor `__init__()` and instance methods, and explain the role of `self`;
- use class variables, class methods and the special methods `__str__()` and `__repr__()`;
- apply encapsulation to protect the data of an object;
- model "has-a" relationships with composition and "is-a" relationships with inheritance;
- override methods, call the parent class with `super()`, and write polymorphic code;
- explain how Python searches for methods under multiple inheritance.

1 What is object-oriented programming?

Object-oriented programming (OOP) is a programming paradigm based on the concept of *objects*, which bundle together data and code: data in the form of *fields* (often called *attributes* or *properties*), and code in the form of procedures (often called *methods*).

A common feature of objects is that methods are attached to them and can access and modify the object's data fields. In this style of OOP, there is usually a special name, such as `this` or `self`, that refers to the current object. Programs are designed as collections of objects that interact with one another. OOP languages are diverse, but the most popular ones are **class-based**: objects are *instances* of *classes*, which also determine their *types*.

Many of the most widely used programming languages, such as C++, Java and Python, are multi-paradigm: they support object-oriented programming to a greater or lesser degree, typically in combination with imperative, procedural programming.

You have been using objects all along

In Python, every value is an object: an integer, a string, a list, a dictionary — even a function. When you write `my_list.append(4)` or `"hello".upper()`, you are calling a *method* of an object. In this lecture, we learn how to define our **own** classes of objects.

Like other languages, an OOP language supports **imperative programming** — built-in data types such as integers, floating-point numbers and strings; data structures such as lists and hash tables (Python's dictionaries); and structured programming constructs such as loops and conditionals — and **modular programming**, the ability to group code into files and modules. Modules are *namespaced*, so that a name defined in one module does not conflict with a function or variable of the same name in another module. Python supports both. On top of these, the following concepts are central to OOP, and each of them has its own section below:

1. classes and objects;
2. class variables and class methods;
3. data abstraction and encapsulation;
4. composition;
5. inheritance and overriding;
6. polymorphism;
7. multiple inheritance.

2 Classes and objects

2.1 Terminology

Languages that support classes are built on two main concepts:

1. **Classes** — the definitions of the data format and the available methods for a given type, or class, of objects. Classes may also contain data and methods themselves.
2. **Objects** — instances of classes.

Objects sometimes correspond to things found in the real world. For example, a graphics program may have objects such as "circle", "square" and "menu", and an online shopping system might have objects such as "shopping cart", "customer" and "product". Sometimes objects represent more abstract entities, such as an open file, or a service that converts measurements from US customary units to metric units. In fact, we have already used many objects in Python: a particular list, a particular tuple, or even a single integer or complex number.

Each object is said to be an **instance** of a particular class; for example, an object whose name field is set to "Mary" might be an instance of the class `Employee`. Procedures in OOP are known as **methods**, and variables are also known as **fields**, **members**, **attributes** or **properties**. This leads to the following terms:

- **class variables** belong to the class as a *whole*; there is only one copy of each of them;
- **instance variables** (or **attributes**) are data that belong to *individual* objects; every object has its own copy of each of them;
- **member variables** refers to both the class and the instance variables defined by a particular class;
- **class methods** belong to the class as a whole and have access only to class variables and to the inputs of the method call;
- **instance methods** belong to *individual* objects and have access to the instance variables of the specific object on which they are called, to their inputs, and to class variables.

Objects are accessed rather like variables with a complex internal structure. They provide a layer of abstraction that separates the internal code of the object from the external code that uses it. External code can use an object by calling one of its instance methods with some input parameters, by reading

an instance variable, or by writing to an instance variable. Objects are created by calling a special method of the class known as the **constructor**. A program may create many instances of the same class, which operate independently of each other — an easy way of applying the same procedures to different sets of data.

2.2 Defining a class

The simplest form of class definition looks like this:

```
class ClassName:
    <statement-1>
    ...
    <statement-N>
```

Class definitions, like function definitions with `def`, must be executed before they have any effect. In practice, the statements inside a class definition are usually function definitions — the **methods** — but other statements are allowed and sometimes useful (for example, to define class variables). By convention (PEP 8), class names use capitalised words, such as `Car` or `ElectricCar`.

2.3 Attribute references and instantiation

A class object supports two kinds of operation: *attribute references* and *instantiation.*

Attribute references use the standard syntax `obj.name`. Valid attribute names are all the names defined in the class. In the next cell, `MyClass.i` and `MyClass.say_hello` are valid attribute references, returning an integer and a function object, respectively. Note that `MyClass.say_hello` only refers to the function; it does not execute it.

Instantiation uses function notation: pretend that the class is a function without parameters that returns a new instance of the class. For example, `x = MyClass()` creates a new instance (object) of the class and assigns it to the variable `x`. We can then call the method on the instance: `x.say_hello()`.

```
class MyClass:
    """A simple example class."""
    i = 12345

    def say_hello(self):
        return "Hello World"

print(MyClass.i)              # an attribute reference: a class variable
print(MyClass.say_hello)      # a function object (not executed!)
```

```
x = MyClass()                # instantiation: create a new object of the
 ↪  class
print(x.say_hello())         # call the method on the instance
```

```
12345
<function MyClass.say_hello at 0x7dbc19ff8880>
Hello World
```

2.4 The constructor `__init__()`

Instantiation ("calling" a class) creates an empty object. Many classes, however, want to create objects with instance variables set to a specific *initial* state. Therefore, a class may define a special method called `__init__()`:

```
def __init__(self):
    self.data = []
```

When a class defines an `__init__()` method, instantiation automatically calls `__init__()` on the newly created object. So, in this example, a new, initialised instance is obtained with `x = MyClass()`, after which `x.data` exists.

The `__init__()` method may of course have further parameters for greater flexibility; the arguments given to the class in the instantiation are passed on to `__init__()`. For example, let us define a class for complex numbers. A complex number has the form $x = a + \mathrm{i}\,b$, where a and b are real numbers and i is the imaginary unit, defined by $\mathrm{i} = \sqrt{-1}$. Since i is always the same, a complex number is completely described by its *real part* a and its *imaginary part* b:

```
class Complex:
    """A complex number a + ib, stored by its real and imaginary parts."""
    def __init__(self, real_part, imag_part):
        self.r = real_part
        self.i = imag_part

x = Complex(3.0, -4.5)
print(x.r, x.i)
```

```
3.0 -4.5
```

(Python actually has a built-in complex type, e.g. `3.0 - 4.5j`, so in practice you would not need this class.)

2.5 Example: a Car class

Throughout this lecture, we practise with two running examples:

- a car, and an electric car as a specific type of car;
- trash, with waste paper and cans as two specific types of trash.

We start with the class `Car`.

```
class Car:
    """Model a car."""

    def __init__(self, make_outside, model_outside):
        """Initialise a Car object.

        The constructor receives the make (the manufacturer) and the model
 ↪  of
        the car from the external code, i.e. from the outside world.
        """
        # We could simply call the parameters "make" and "model": self.make
         ↪  is
        # clearly different from make. The suffix "_outside" just
         ↪  emphasises
        # where the values come from.
        self.make = make_outside
        self.model = model_outside
        self.tank_cap = 60  # a 60-litre tank
        self.tank = 0       # the tank is initially empty
        print("Constructor of Car executed: {make} -
         ↪  {model}".format(make=self.make, model=self.model))

    def fill_up(self):
        """Fill the tank to the brim."""
        self.tank = self.tank_cap
        print("Fill-up done: full!")

    # other methods would follow here, e.g.
    # def other_method(self, ...):
```

```
# Now we can create (instantiate) an object of type Car ...
my_car = Car("Nissan", "Micra")
# ... and access all its attributes.
```

```
print(my_car.make)                                  # self.make in the class
 ↪  definition
print(my_car.model)                                 # self.model
print("Tank in the initial state =", my_car.tank)    # self.tank
print(my_car.tank_cap)                              # self.tank_cap
```

```
Constructor of Car executed: Nissan - Micra
Nissan
Micra
Tank in the initial state = 0
60
```

Note that external code can read (and change) all the data of a `Car` instance. Unlike C++ and Java, Python does not enforce private data members or private methods: everything can be seen and accessed from outside. We return to this point in the section on encapsulation.

```
# Of course, we can create another Car with different attributes.
your_car = Car("Nissan", "Leaf")
print(your_car.make)
print(your_car.model)
# fill_up() has no return statement, so it returns None, which print()
 ↪  displays.
print(your_car.fill_up())
print("Tank after the fill-up =", your_car.tank)
```

```
Constructor of Car executed: Nissan - Leaf
Nissan
Leaf
Fill-up done: full!
None
Tank after the fill-up = 60
```

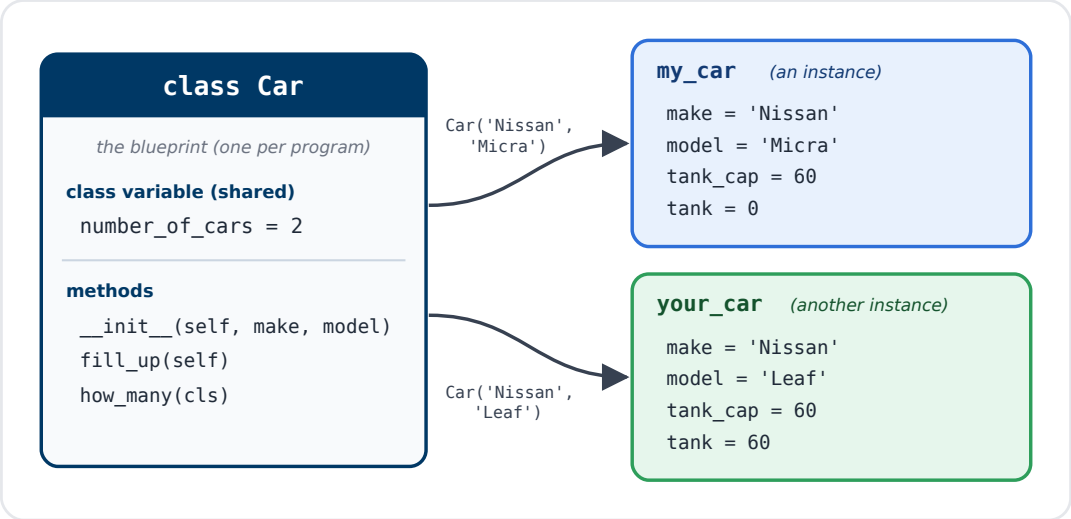


Figure 1: The class `Car` is the blueprint; `my_car` and `your_car` are two independent instances, each with its own instance variables. The class variable `number_of_cars` and the class method `how_many()` are added in the section on class variables.

```
print(my_car)    # not very informative -- we improve this below
```

```
<__main__.Car object at 0x7dbc19fa16a0>
```

2.6 Understanding `self`

All the methods defined in `Car` have a first parameter called `self`. Now is the right time to explain what it means.

When we write a class, none of its objects exist yet. Later, there may be many of them — `my_car`, `your_car`, and so on — and every method must be able to work on whichever object it is called on. Inside the class definition, we cannot write `my_car.tank`, because the class does not (and should not) know which variable names the external code will use. The first parameter of each method solves this problem: **`self` is the particular object on which the method is called**. When you write

```
my_car.fill_up()
```

Python automatically translates this into

```
Car.fill_up(my_car)
```

so that, inside `fill_up()`, `self` refers to `my_car`, and `self.tank` means "the tank of *this particular* car". When you call `your_car.fill_up()`, the same code runs with `self` referring to `your_car`. (The name `self` is only a convention, but you should always follow it.)

```
class MyDumbDataType:
    def __init__(self, data):
        self.data = data

    def write_data_out(self):
        print(self.data)

x = MyDumbDataType("Hello World")
y = MyDumbDataType("Hello Earth")
x.write_data_out()  # inside the method, self is x
y.write_data_out()  # inside the method, self is y
# exactly what x.write_data_out() does behind the scenes
MyDumbDataType.write_data_out(x)
```

```
Hello World
Hello Earth
Hello World
```

The same holds for the `Car` class. The next two cells call `__init__()` and `fill_up()` directly — something you would not normally do — just to show the role of `self`.

```
x = Car("Audi", "Really Cheap")
# __init__(self, make, model) was called AUTOMATICALLY right after the
 ↪  object was created.
print(x.make, "-", x.model)

# We can call __init__() again through x. This does not create a new
 ↪  object;
# it merely re-assigns the attributes of the existing object x.
x.__init__("BMW", "Terribly Cheap")
print(x.make, "-", x.model)

# The same, written through the class: x is passed explicitly as self.
# (Don't write code like this; it is only for illustration.)
Car.__init__(x, "Mercedes", "Ridiculously Cheap")
print(x.make, "-", x.model)
```

```
Constructor of Car executed: Audi - Really Cheap
Audi - Really Cheap
Constructor of Car executed: BMW - Terribly Cheap
BMW - Terribly Cheap
Constructor of Car executed: Mercedes - Ridiculously Cheap
Mercedes - Ridiculously Cheap
```

```
x.fill_up()                    # the normal way: x is passed as self
 ↪  automatically
print(x.tank)

x.tank = 0                     # reset the tank to zero
print("Tank reset: x.tank =", x.tank)

Car.fill_up(x)  # the same, with self passed explicitly (don't do this!)
print(x.tank)
```

```
Fill-up done: full!
60
Tank reset: x.tank = 0
Fill-up done: full!
60
```

2.7 Printing objects: `__str__()` and `__repr__()`

Printing `my_car` above gave something like `<__main__.Car object at 0x...>`, which is not very informative. To control how an object is displayed, a class can define two more special methods:

- `__str__()` returns a readable string; it is used by `print()` and `str()`;
- `__repr__()` returns an unambiguous representation, ideally one that looks like the code that creates the object; it is used when the object is displayed in Jupyter or inside a list, for example.

Methods whose names begin and end with double underscores, such as `__init__()`, `__str__()` and `__repr__()`, are called **special methods** (or "dunder" methods); Python calls them automatically in particular situations.

```
class Point:
    """A point in the plane."""
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):                # used by print() and str()
        return f"Point at ({self.x}, {self.y})"

    def __repr__(self):  # used e.g. when the object is shown inside a list
        return f"Point({self.x!r}, {self.y!r})"

p = Point(1.5, -2)
```

```
print(p)
print([p, Point(0, 0)])
```

```
Point at (1.5, -2)
[Point(1.5, -2), Point(0, 0)]
```

In an f-string, `!r` inserts the `repr()` of a value instead of its `str()`; for a string, this includes the quotation marks.

3 Class variables and class methods

As mentioned above, a class can have its own state, and hence its own attributes and methods. A **class variable** is defined in the class itself, outside any method, and is meant to be shared by all instances and modified only at the class level, not through an individual instance. There is no need to create an object to use it. Similarly, a **class method** belongs to the class and works at the class level; it, too, can be used without creating any object.

To define a class method, we write the **decorator** `@classmethod` on the line before the method definition. The first parameter of a class method refers to the class itself and is conventionally called `cls`, short for *class*.

Syntax

```
class ClassName:
    class_variable_1 = 0        # any value
    class_variable_2 = 0
    ...

    @classmethod
    def class_method(cls):
        <statement-1>
        ...
        <statement-N>
```

Example. We extend our class `Car` with a class variable `number_of_cars`, which keeps track of how many `Car` instances have been created. The variable starts at zero and increases by one every time a new `Car` is created. Note that this class variable exists even before any car has been created: in that case, `number_of_cars` is simply zero. We also define a class method `how_many()`, which prints how many cars have been created.

```
class Car:
    """Model a car, and count how many cars have been created."""

    number_of_cars = 0              # a class variable, shared by all cars

    def __init__(self, make, model):
        """Initialise a Car object with its make and model."""
        Car.number_of_cars += 1  # one more car exists now
        self.make = make
        self.model = model
        self.tank_cap = 60       # a 60-litre tank
        self.tank = 0            # the tank is initially empty
        print("Constructor of Car executed: {make} -
         ↪  {model}".format(make=self.make, model=self.model))

    def fill_up(self):
        """Fill the tank to the brim."""
        self.tank = self.tank_cap
        print("Fill-up done: full!")

    @classmethod
    def how_many(cls):              # cls plays the role of "self" for the
     ↪  class
        print("Number of cars =
         ↪  {number}".format(number=cls.number_of_cars))
```

```
# External code. Even before any Car instance exists, we can ask how many
# cars have been created -- and the answer is, of course, zero.
print('Number of cars (class variable) =', Car.number_of_cars)
print("--------------------------------")

my_car = Car("Nissan", "Micra")
print('Number of cars (class variable) =', Car.number_of_cars)
Car.how_many()
print("--------------------------------")

your_car = Car("Mercedes", "A 160")
print('Number of cars (class variable) =', Car.number_of_cars)
Car.how_many()
print("--------------------------------")
```

```
his_car = Car("Mercedes", "A 140")
print('Number of cars (class variable) =', Car.number_of_cars)
Car.how_many()
```

```
Number of cars (class variable) = 0
--------------------------------
Constructor of Car executed: Nissan - Micra
Number of cars (class variable) = 1
Number of cars = 1
--------------------------------
Constructor of Car executed: Mercedes - A 160
Number of cars (class variable) = 2
Number of cars = 2
--------------------------------
Constructor of Car executed: Mercedes - A 140
Number of cars (class variable) = 3
Number of cars = 3
```

Alternative constructors. A class method can also create objects. Since its first parameter `cls` refers to the class itself, `cls(...)` works exactly like the class name followed by brackets: it creates a new instance. In the following example, the class `Pizza` has a constructor `__init__(self, ingredients)` and a class method `margherita()`, which creates one particular type of pizza. Inside it, we write `cls(["mozzarella", "tomatoes"])` instead of `Pizza(["mozzarella", "tomatoes"])`: you can imagine that `cls` is literally replaced by `Pizza`. Such class methods are called **factory methods** or **alternative constructors**; you will meet them in libraries, e.g. `dict.fromkeys()`.

```
class Pizza:
    def __init__(self, ingredients):
        self.ingredients = ingredients

    def __repr__(self):
        """Represent the object, e.g. when it is printed."""
        return f'Pizza({self.ingredients!r})'

    @classmethod                    # the following method is a class method
    def margherita(cls):
        return cls(["mozzarella", "tomatoes"])
```

```
# External code, outside the class Pizza
my_pizza = Pizza(["cheese", "cheese"])
print(my_pizza)
your_pizza = Pizza.margherita()
```

```
print(your_pizza)
```

```
Pizza(['cheese', 'cheese'])
Pizza(['mozzarella', 'tomatoes'])
```

(Since `Pizza` defines `__repr__()` but not `__str__()`, `print()` falls back on `__repr__()`.)

4 Data abstraction and encapsulation

4.1 Data abstraction

Data abstraction is a design principle whereby data are visible only to semantically related functions, in order to prevent misuse. Its success has made **data hiding** a common design principle in object-oriented and functional programming.

If a class does not allow external code to access the internal data of its objects directly, and permits access only through methods, this is a strong form of abstraction known as **information hiding**. Some languages (Java, for example) let classes enforce access restrictions explicitly: internal data are marked with the keyword `private`, while methods intended for use by external code are marked `public`. Methods may also be public, private or have an intermediate level such as `protected`, which allows access from the same class and its subclasses, but not from objects of other classes.

In other languages, such as Python, this is enforced only **by convention**: names that start with an underscore, such as `_tank`, are meant for internal use only, and well-behaved external code does not touch them. Nothing technically prevents misuse, however:

```
# In Python, the data of an object are easily accessible ... and easily
 ↪  misused.
my_car.fill_up()
my_car.tank = my_car.tank + 5           # put in some more fuel
print("Current tank =", my_car.tank)    # clearly wrong: more than the
 ↪  capacity!
print("Tank capacity =", my_car.tank_cap)
```

```
Fill-up done: full!
Current tank = 65
Tank capacity = 60
```

4.2 Encapsulation

Encapsulation prevents external code from being concerned with the internal workings of an object. This makes code easier to change (refactor): for example, the author of a class can change how its objects store their data internally without changing any external code, as long as the "public" methods still work in the same way. Encapsulation also encourages programmers to put all the code concerned with a certain set of data in the same class, which makes it easier for other programmers to understand. Encapsulation thus promotes **decoupling**: the parts of a program depend on each other as little as possible.

Hiding the data members may look complicated and unnecessary, but it turns out to be needed very often in real-world problems. In many situations, data members should only take values within a given range, or only of a particular type. For example, if a car has a tank capacity of 60 litres, adding 50 litres to a tank that already holds 20 litres would overfill it. External code might also accidentally "fill" the tank with a string instead of a number. Such operations are invalid and should raise an error or at least notify the user. In the worst case, no obvious error occurs when the data member is modified; instead, the invalid value silently spreads through the program and spoils the final results.

Hiding the data members and letting external code access or modify them only through methods makes a class longer and somewhat more complicated. In the long run, however, this effort usually saves a lot of headaches when developing a sophisticated program.

Example. We extend the class `Car` once more, with a method `add_fuel(self, amount)` that checks whether the tank would overflow *before* adding the fuel. We also add a class variable `fuel_type`, which we will need later.

```
class Car:
    """Model a car, count the cars created, and protect the fuel tank."""

    number_of_cars = 0
    fuel_type = 'petrol'

    def __init__(self, make, model):
        """Initialise a Car object with its make and model."""
        self._odometer = 0  # the leading underscore says: internal use
         ↪  only
        self.make = make
        self.model = model
        self.tank_cap = 60  # a 60-litre tank
        self.tank = 0       # the tank is initially empty
        print("Constructor of Car executed: {make} -
         ↪  {model}".format(make=self.make, model=self.model))
        Car.number_of_cars += 1
```

```
    def fill_up(self):
        """Fill the tank to the brim."""
        self.tank = self.tank_cap
        print("Fill-up done: full!")

    def add_fuel(self, amount):
        """Add fuel to the tank.

        The fuel in the tank must not exceed the tank capacity. This method
        checks the condition first; if the tank would overflow, it does not
        add the fuel but notifies the user instead.
        """
        new_tank = self.tank + amount
        if new_tank <= self.tank_cap:
            self.tank = new_tank
        else:
            print("The tank is not big enough!")
```

```
my_car = Car("Mercedes", "A 160")
print("--------------------------")
print("Add 50 litres to my car.")
my_car.add_fuel(50)
print("my_car.tank =", my_car.tank)

print("Add another 70 litres. Does it work?")
my_car.add_fuel(70)
print("my_car.tank =", my_car.tank)
```

```
Constructor of Car executed: Mercedes - A 160
--------------------------
Add 50 litres to my car.
my_car.tank = 50
Add another 70 litres. Does it work?
The tank is not big enough!
my_car.tank = 50
```

Going further: properties

> **Optional**
>
> Python offers an elegant way of combining the convenience of `obj.attribute` with the safety of a method: **properties**. With the decorator `@property`, reading `tank.level` actually calls a method; with `@level.setter`, *assigning* to `tank.level` calls another method, which can check the new value.

```
class Tank:
    """A fuel tank whose level can never exceed its capacity."""
    def __init__(self, capacity):
        self.capacity = capacity
        self._level = 0                  # "private" by convention

    @property
    def level(self):                     # called when we READ tank.level
        return self._level

    @level.setter
    def level(self, value):              # called when we WRITE tank.level
     ↪  = value
        if 0 <= value <= self.capacity:
            self._level = value
        else:
            print(f"Rejected: {value} L is outside the range
             ↪  0-{self.capacity} L.")

tank = Tank(60)
tank.level = 50      # accepted
tank.level = 65      # rejected by the setter
print(tank.level)
```

```
Rejected: 65 L is outside the range 0-60 L.
50
```

5 Composition

Objects can contain other objects in their instance variables; this is known as **object composition**. For example, an object of the class `Employee` might contain an object of the class `Address`, in addition to its own instance variables such as `first_name` and `position`. Composition represents **"has-a"**

relationships: every employee *has an* address, so every `Employee` object has a place to store an `Address` object.

```
class Address:
    def __init__(self, street, city, postcode):
        self.street = street
        self.city = city
        self.postcode = postcode

    def __str__(self):
        return f"{self.street}, {self.city} {self.postcode}"

class Employee:
    def __init__(self, first_name, position, address):
        self.first_name = first_name
        self.position = position
        self.address = address          # an Employee HAS an Address

mary = Employee("Mary", "engineer", Address("University Avenue", "Glasgow",
 ↪  "G12 8QQ"))
print(mary.first_name, "works as an", mary.position)
print("Address:", mary.address)    # uses Address.__str__()
print("City:", mary.address.city)  # reach into the contained object
```

```
Mary works as an engineer
Address: University Avenue, Glasgow G12 8QQ
City: Glasgow
```

6 Inheritance

Languages that support classes almost always support **inheritance**. Inheritance allows classes to be arranged in a hierarchy that represents **"is-a-type-of"** relationships. For example, the class `Employee` might inherit from the class `Person`. All the data and methods available in the parent class also appear in the child class, with the same names. For example, the class `Person` might define the variables `first_name` and `last_name` and a method `make_full_name()`; these will also be available in the class `Employee`, which might add the variables `position` and `salary`. This technique makes it easy to reuse the same procedures and data definitions, and it can mirror real-world relationships in an intuitive way: rather than database tables and subroutines, the developer works with objects that users are familiar with — objects from their application domain.

If class `A` is a type of class `B`, we let class `A` inherit from class `B`; we also say that class `A` is **derived from** class `B`. We call

- class `B` the **base class**, **superclass** or **parent class**;
- class `A` the **derived class**, **subclass** or **child class**.

The names are self-explanatory if you think of the parent–child relationship.

Syntax

```
class DerivedClassName(BaseClassName):
    <statement-1>
    ...
    <statement-N>
```

The base class must be defined before the derived class: Python must know `BaseClassName` before it can define a child of it.

Example. Let us describe the idea of *trash*, together with two particular types of trash: *waste paper* and *cans*. Clearly, waste paper and cans are both types of trash, so it is reasonable to model the following relationships:

- `WastePaper` is derived from `Trash`;
- `Can` is derived from `Trash`.

`Trash` has two attributes, `weight` and `value`, and two methods, `shred()` and `burn()`. `WastePaper` and `Can` inherit all of these from their base class `Trash`. In addition, `WastePaper` has three more attributes, `condition`, `value_wet` and `value_dry`, and one more method, `value_refresh()`, while `Can` has one more method, `crush()`. The child classes call the constructor of the parent class with `super().__init__(...)`, so that the inherited attributes are initialised too.

```
class Trash:
    def __init__(self, weight, value):
        """Initialise a Trash instance."""
        self.weight = weight
        self.value = value
        print("Trash constructor: Trash({0}, {1})".format(self.weight,
        ↪  self.value))

    def shred(self):
        print("In Trash: Shred me, please!")

    def burn(self):
        print("In Trash: Burn me, please!")

class WastePaper(Trash):
    def __init__(self, weight, value, condition, value_wet, value_dry):
```

```
        super().__init__(weight, value)  # initialise the Trash part first
        self.condition = condition
        self.value_wet = value_wet
        self.value_dry = value_dry
        print("WastePaper constructor: WastePaper({0}, {1}, {2}, {3},
        ↪  {4})".format(
            self.weight, self.value, self.condition, self.value_wet,
            ↪  self.value_dry))

    def value_refresh(self):
        self.value_wet = 0
        self.value_dry = 0

class Can(Trash):
    def __init__(self, weight, value):
        super().__init__(weight, value)
        print("Can constructor: Can({0}, {1})".format(weight, value))

    def crush(self):
        print("In Can: Crush me, please!")
```

```
# External code, outside the classes above
my_trash = Trash(0.1, 1)
print("-----------------------------------")

# The constructor of WastePaper first calls the constructor of Trash
# explicitly; then the rest of WastePaper.__init__() is executed.
my_waste_paper = WastePaper(0.2, 1, "new", 0.8, 0.2)

# The class WastePaper does not define the attributes "weight" and "value",
# or the method shred(), itself: they are inherited from the parent class
↪  Trash.
print("my_waste_paper.weight =", my_waste_paper.weight)
print("my_waste_paper.value =", my_waste_paper.value)
my_waste_paper.shred()
print("-----------------------------------")

# Similarly, the constructor of Trash is called inside the constructor of
↪  Can.
my_can = Can(0.2, 2.0)
my_can.crush()
```

```
Trash constructor: Trash(0.1, 1)
---------------------------------
Trash constructor: Trash(0.2, 1)
WastePaper constructor: WastePaper(0.2, 1, new, 0.8, 0.2)
my_waste_paper.weight = 0.2
my_waste_paper.value = 1
In Trash: Shred me, please!
---------------------------------
Trash constructor: Trash(0.2, 2.0)
Can constructor: Can(0.2, 2.0)
In Can: Crush me, please!
```

6.1 Overriding methods and `super()`

One of the most important and exciting features of inheritance is that subclasses can **override** the methods defined by their superclass: if a subclass defines a method with the same name as a method of its parent, the subclass's version is used for its objects. Inside the overriding method, `super().method_name()` can still call the parent's version, so that the subclass *extends* rather than replaces the parent's behaviour.

In the following example, we use `print()` to track which method is called and where it is executed. The example has no use other than to help you understand inheritance and the mechanism of overriding. This time, `Trash` also defines `crush()`. `WastePaper` and `Can` override it, `GoldDummyCan` extends it with `super()`, and the plain `Trash` keeps the original version.

```
class Trash:
    def __init__(self, weight, value):
        """Initialise a Trash instance."""
        self.weight = weight
        self.value = value
        print("Trash constructor: Trash({0}, {1})".format(self.weight,
        ↪  self.value))

    def shred(self):
        print("In Trash: Shred me, please!")

    def burn(self):
        print("In Trash: Burn me, please!")

    def crush(self):
        print("In Trash: Crush me, please!")

class WastePaper(Trash):
```

```
    def __init__(self, weight, value, condition, value_wet, value_dry):
        super().__init__(weight, value)
        self.condition = condition
        self.value_wet = value_wet
        self.value_dry = value_dry
        print("WastePaper constructor: WastePaper({0}, {1}, {2}, {3},
         ↪  {4})".format(
            self.weight, self.value, self.condition, self.value_wet,
             ↪  self.value_dry))

    def value_refresh(self):
        self.value_wet = 0
        self.value_dry = 0

    def crush(self):                        # overrides Trash.crush()
        print("In WastePaper: Crush me, please!")

class Can(Trash):
    def __init__(self, weight, value):
        super().__init__(weight, value)
        print("Can constructor: Can({0}, {1})".format(weight, value))

    def crush(self):                        # overrides Trash.crush()
        print("In Can: Crush me, please!")

class GoldDummyCan(Trash):
    def __init__(self, weight, value):
        super().__init__(weight, value)
        print("GoldDummyCan constructor: GoldDummyCan({0}, {1}) -- I am
         ↪  rich too!".format(weight, value))

    def crush(self):                        # extends Trash.crush()
        super().crush()                     # first do what the parent does
         ↪  ...
        print("In GoldDummyCan: Crush me, please!")   # ... then something
         ↪  extra
```

```
# External code, outside the classes above
my_trash = Trash(0.1, 1)
my_trash.crush()
```

```
print("-----------------------------------")
my_can = Can(0.2, 2.0)
my_can.crush()

print("-----------------------------------")
his_waste_paper = WastePaper(0.02, 0.2, "old", 0.05, 0.15)
his_waste_paper.crush()

print("-----------------------------------")
your_can = GoldDummyCan(0.05, 200)
your_can.crush()
```

```
Trash constructor: Trash(0.1, 1)
In Trash: Crush me, please!
-----------------------------------
Trash constructor: Trash(0.2, 2.0)
Can constructor: Can(0.2, 2.0)
In Can: Crush me, please!
-----------------------------------
Trash constructor: Trash(0.02, 0.2)
WastePaper constructor: WastePaper(0.02, 0.2, old, 0.05, 0.15)
In WastePaper: Crush me, please!
-----------------------------------
Trash constructor: Trash(0.05, 200)
GoldDummyCan constructor: GoldDummyCan(0.05, 200) -- I am rich too!
In Trash: Crush me, please!
In GoldDummyCan: Crush me, please!
```

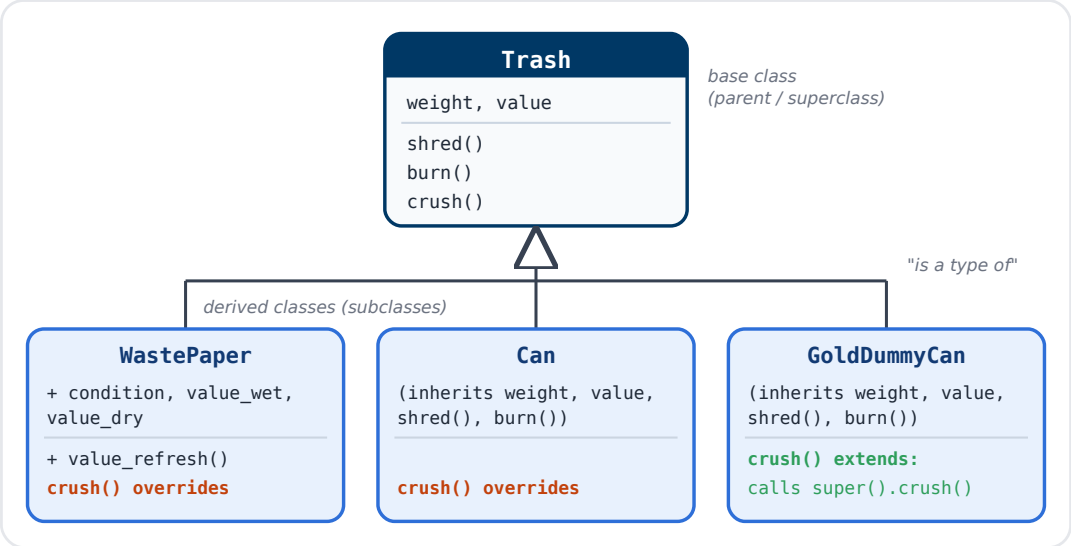


Figure 2: The `Trash` class hierarchy. `WastePaper` and `Can` override `crush()`; `GoldDummyCan` extends it by calling `super().crush()` first.

Another example: an electric car. We define a class `ElectricCar` derived from the latest version of `Car`. An electric car has a battery capacity and a charge level instead of a tank capacity and a tank level, so it needs charging rather than refuelling. A first attempt reveals a problem:

```
class ElectricCar(Car):
    """Model an electric car."""
    def __init__(self, make, model):
        """Initialise an electric car."""
        super().__init__(make, model)
        self.battery_cap = 30     # kWh
        self.charge = 0
        print("ElectricCar constructor!")

    def charge_full(self):
        """Charge the battery fully."""
        self.charge = self.battery_cap
        print("Charged: full!")

# External code
my_electric_car = ElectricCar("Nissan", "Leaf")
my_electric_car.charge_full()
my_electric_car.fill_up()         # oops: this still works, although it
 ↪  should not
print(my_electric_car.fuel_type)  # still petrol???
```

```
Constructor of Car executed: Nissan - Leaf
ElectricCar constructor!
Charged: full!
Fill-up done: full!
petrol
```

The electric car has inherited `fill_up()` and `fuel_type = 'petrol'` from `Car`. We fix this by **overriding** both the class variable and the method in the child class:

```
class ElectricCar(Car):
    """Model an electric car."""
    fuel_type = 'electricity'          # overrides Car.fuel_type

    def __init__(self, make, model):
        """Initialise an electric car."""
        super().__init__(make, model)
        self.battery_cap = 30          # kWh
        self.charge = 0
        print("ElectricCar constructor!")

    def charge_full(self):
        """Charge the battery fully."""
        self.charge = self.battery_cap
        print("Charged: full!")

    def fill_up(self):                 # overrides Car.fill_up()
        """Refuelling is not possible for an electric car."""
        print("No fuel tank!")

# External code
my_electric_car = ElectricCar("Nissan", "Leaf")
my_electric_car.charge_full()
my_electric_car.fill_up()              # now the overriding method is called
print(my_electric_car.fuel_type)       # and the fuel type is now correct
```

```
Constructor of Car executed: Nissan - Leaf
ElectricCar constructor!
Charged: full!
No fuel tank!
electricity
```

7 Polymorphism

Polymorphism (from the Greek for "many shapes") means that code can work with objects of different classes in a hierarchy — the parent class or any of its descendants — without knowing which class each object actually belongs to. The same method name may then behave differently depending on the class of the object.

For example, the classes `Circle` and `Rectangle` may be derived from a common class `Shape`. Each type of shape implements its own `draw()` method, while the code that calls `draw()` can remain indifferent to the particular type of shape being drawn. This is another type of abstraction: it simplifies the code outside the class hierarchy and enables a strong separation of concerns.

Example 1: crushing trash. The function below calls `crush()` on whatever trash it receives. Each object automatically uses its own version of the method.

```
def crush_random_trash(random_trash):
    random_trash.crush()       # works for ANY kind of trash

trash_list = [Trash(0.1, 1),
              WastePaper(0.1, 1, "relatively new", 0.3, 0.7),
              Can(0.1, 1.5),
              GoldDummyCan(0.01, 250)]
print("-------------------------")
for trash in trash_list:
    crush_random_trash(trash)
    print("-------------------------")
```

```
Trash constructor: Trash(0.1, 1)
Trash constructor: Trash(0.1, 1)
WastePaper constructor: WastePaper(0.1, 1, relatively new, 0.3, 0.7)
Trash constructor: Trash(0.1, 1.5)
Can constructor: Can(0.1, 1.5)
Trash constructor: Trash(0.01, 250)
GoldDummyCan constructor: GoldDummyCan(0.01, 250) -- I am rich too!
-------------------------
In Trash: Crush me, please!
-------------------------
In WastePaper: Crush me, please!
-------------------------
In Can: Crush me, please!
-------------------------
In Trash: Crush me, please!
In GoldDummyCan: Crush me, please!
-------------------------
```

Example 2: a mixed garage. The loop below treats petrol and electric cars alike; each car responds to `fill_up()` in its own way.

```
garage = [Car("Toyota", "Corolla"), ElectricCar("Nissan", "Leaf")]
print("-------------------------")
for car in garage:
    print(car.make, car.model, "runs on", car.fuel_type, end=" -> ")
    car.fill_up()
```

```
Constructor of Car executed: Toyota - Corolla
Constructor of Car executed: Nissan - Leaf
ElectricCar constructor!
-------------------------
Toyota Corolla runs on petrol -> Fill-up done: full!
Nissan Leaf runs on electricity -> No fuel tank!
```

8 Multiple inheritance

Another important concept related to inheritance is **multiple inheritance**: a derived (child) class may have several base (parent) classes. Python supports a form of multiple inheritance. Not all languages do: Java and C#, for example, allow a class to inherit from only one parent class (they offer *interfaces* instead), whereas C++ and Python allow several.

Syntax

```
class DerivedClassName(Base1, Base2, Base3):
    <statement-1>
    ...
    <statement-N>
```

The search path for methods and attributes. If class `A` is derived from a single base class `B`, the search path is easy to deduce. Suppose that `B` defines a method `do_something()`. If `A` overrides it with its own version, an object `obj_A` of class `A` uses `A`'s version. If `A` does not define `do_something()` itself, Python searches upwards, finds the method in `B` and executes it: `obj_A.do_something()` runs the method defined in `B`. The same applies to attributes.

With multiple inheritance, the search path becomes a little more complicated. Suppose that `DerivedClassName` is derived from `Base1`, `Base2` and `Base3`, as in the syntax above. If a method is not defined in `DerivedClassName` itself, in which base class should Python look for it? In the simplest cases, you can think of the search as **depth-first** and **left-to-right**, never searching the same class twice where the hierarchy overlaps. So, if an attribute is not found in `DerivedClassName`, it is searched for in `Base1`, then (recursively) in the base classes of `Base1`; if it is not found there, it is

searched for in `Base2`, and so on. The resulting order is called the **method resolution order** (MRO), and you can display it with `ClassName.mro()`.

In fact, the rule is slightly more complex than that: the method resolution order is computed so as to support cooperative calls to `super()`. This is necessary because every case of multiple inheritance involves one or more *diamond* relationships, in which at least one parent class can be reached along several paths from the bottom class. For example, all classes in Python inherit from the class `object`, so any case of multiple inheritance provides more than one path to `object`. We will not delve into this topic any further; interested students can find the details in good books on Python and object-oriented programming.



Figure 3: No regrets cat

Example. A *tigon* is the offspring of a male tiger and a female lion. We model it with a class `Tigon` derived from both `Tiger` and `Lion`.

```
class Tiger:
    def __init__(self, name):
        self.name = name
        print("A tiger is born with the name {0}.".format(self.name))

    def run(self):
        print("I run like a tiger.")

    def eat(self, food):
        print("I eat " + food + " like a tiger.")

    def hunt(self, prey):
        print("I hunt " + prey + " alone.")

    def meow(self):
        print("MEOW")

class Lion:
    def __init__(self, name):
        self.name = name
        print("A lion is born with the name {0}.".format(self.name))

    def run(self):
        print("I run like a lion.")

    def eat(self, food):
        print("I eat " + food + " like a lion.")

    def hunt(self, prey):
        print("I hunt " + prey + " with my pride.")

    def roar(self):
        print("GRRR")
```

```
class Tigon(Tiger, Lion):
    def __init__(self, name):
        self.name = name
```

```
        print("My father is a tiger and my mother is a lion. My name is
        ↪ {}.".format(self.name))

    def run(self):
        print("I run more slowly than both a tiger and a lion.")
```

```
mary = Tiger("Mary")
tom = Lion("Tom")
jerrom = Tigon("Jerrom")
```

```
A tiger is born with the name Mary.
A lion is born with the name Tom.
My father is a tiger and my mother is a lion. My name is Jerrom.
```

```
mary.meow()
tom.roar()

jerrom.roar()            # only Lion defines roar()
jerrom.meow()            # only Tiger defines meow()
jerrom.run()             # Tigon's own version overrides both parents
jerrom.eat("meat")       # both parents define eat(): Tiger comes first in
↪ the MRO
jerrom.hunt("a deer")    # the same for hunt()
print(Tigon.mro())
```

```
MEOW
GRRR
GRRR
MEOW
I run more slowly than both a tiger and a lion.
I eat meat like a tiger.
I hunt a deer alone.
[<class '__main__.Tigon'>, <class '__main__.Tiger'>, <class '__main__.Lion'>, <class
↪ 'object'>]
```

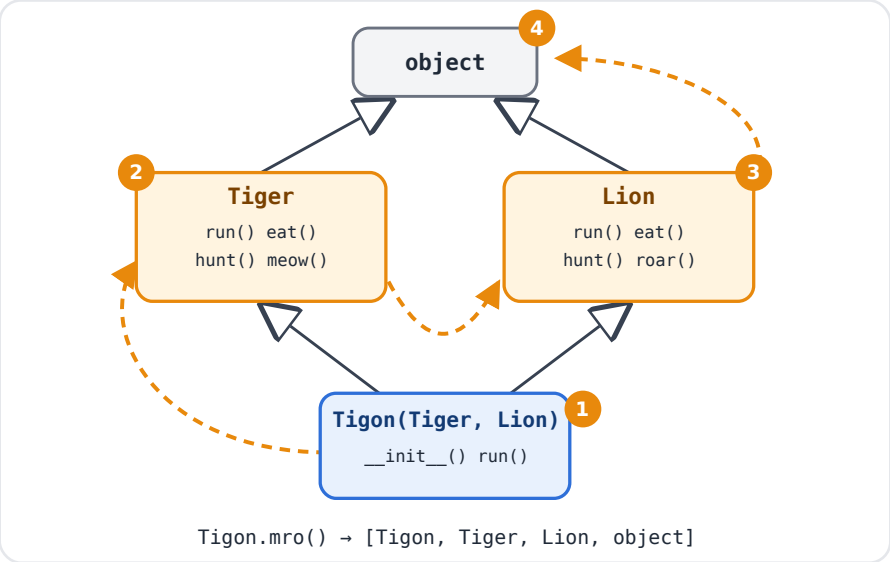


Figure 4: The method resolution order of `Tigon`. Python looks for a method first in `Tigon` itself, then in `Tiger`, then in `Lion` and finally in `object`.

Summary

- A **class** is a blueprint; an **object** (instance) is created by calling the class, which runs the constructor `__init__()`. Instance variables are stored on `self`, which is the object on which a method is called.
- **Class variables** are shared by all instances; **class methods** (`@classmethod`) receive the class as `cls` and can serve as alternative constructors. `__str__()` and `__repr__()` control how objects are displayed.
- **Encapsulation** protects an object's data by letting external code change it only through methods (or properties) that check the values. In Python, a leading underscore marks internal names by convention.
- **Composition** models "has-a" relationships; **inheritance** models "is-a-type-of" relationships. Subclasses can **override** methods and call the parent's version with `super()`.
- **Polymorphism** lets the same code work with objects of different classes in a hierarchy.
- With **multiple inheritance**, Python searches for methods in the order given by `ClassName.mro()`.

Exercises

Exercise 1: A `Rectangle` class

Write a class `Rectangle` whose constructor receives a `width` and a `height`. Give it the methods `area()`, `perimeter()` and `is_square()`, and a `__str__()` method that returns, for example, `"Rectangle 3 x 4"`. Create a list of several rectangles, print each of them together with its area, and compute their total area.

Exercise 2: Shapes and polymorphism

Write a base class `Shape` with an attribute `name`, a method `area()` that returns `0.0`, and a method `describe()` that prints the name and the area of the shape (by calling `self.area()`). Then derive the classes `Circle` (with a `radius`) and `Rectangle` (with a `width` and a `height`) from `Shape`, and derive `Square` from `Rectangle`, so that `Square(side)` calls `super().__init__(side, side)`. Override `area()` where necessary. Finally, put several different shapes in a list and call `describe()` on each of them. Which version of `area()` is called inside `describe()`, and why?

Exercise 3: Driving the car

Extend the latest `Car` class with an attribute `consumption` (fuel consumption in litres per 100 km) and a method `drive(distance_km)` that reduces the fuel in the tank accordingly and increases the odometer. If there is not enough fuel for the whole trip, the method should not drive at all; instead, it should print how far the car could go with the remaining fuel. Then override `drive()` in `ElectricCar` so that it uses the battery charge and a consumption in kWh per 100 km. Test both classes in a polymorphic loop.

Exercise 4: Composition — a cantilever beam

Write a class `Material` with the attributes `name`, `youngs_modulus` E (in Pa) and `density` ρ (in kg/m³). Then write a class `Beam` for a rectangular cantilever beam that *has a* `Material`, with the attributes `length` L, `width` b and `height` h (in m), and the methods

- `mass()`, which returns $\rho \, L \, b \, h$;
- `second_moment()`, which returns the second moment of area $I = b h^3 / 12$;
- `tip_deflection(force)`, which returns the deflection $\delta = F L^3 / (3 E I)$ under a force F at the free end.

Compare a steel beam ($E = 200$ GPa, $\rho = 7850$ kg/m³) with an aluminium beam ($E = 69$ GPa, $\rho = 2700$ kg/m³), both with $L = 2$ m, $b = 0.05$ m and $h = 0.1$ m, under a force of $F = 1000$ N. Print the mass and the tip deflection (in mm) of each beam with suitable formatting.

Further reading

- The official Python tutorial: Classes.
- Real Python, Object-Oriented Programming (OOP) in Python.